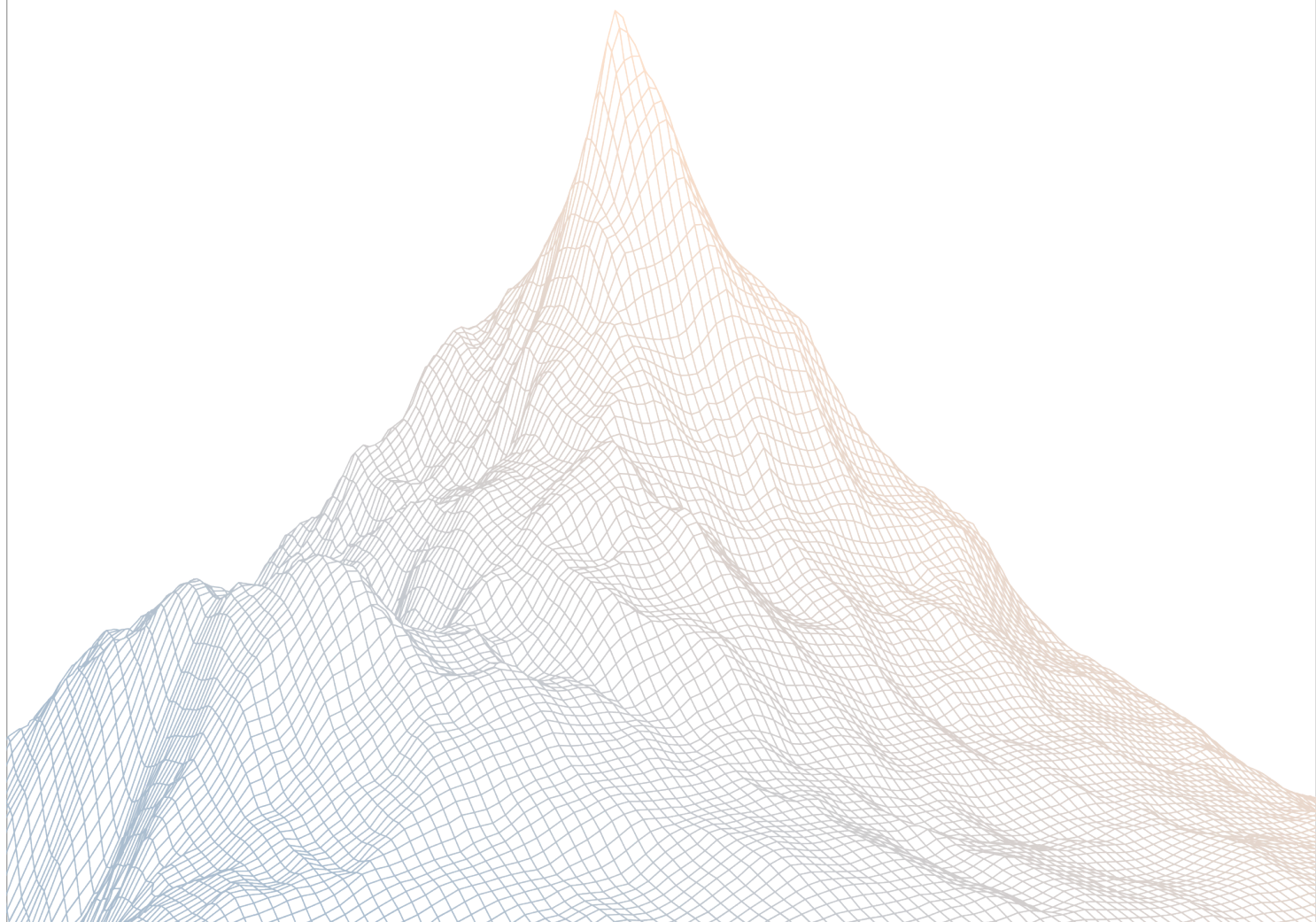


GMX Solana

Smart Contract Security Assessment

VERSION 1.1



AUDIT DATES:

July 15th to July 17th, 2026

AUDITED BY:

Mario Ponder
peakbolt

Contents

1	Introduction	3
1.1	About Zenith	3
1.2	Disclaimer	3
1.3	Risk Classification	3
2	Executive Summary	4
2.1	About GMX Solana Protocol	4
2.2	Scope	4
2.3	Audit Timeline	5
2.4	Issues Found	5
3	Findings Summary	6
4	Findings	7
4.1	Medium Risk	7
4.2	Low Risk	12
4.3	Informational	30

1

Introduction

1.1 About Zenith

Zenith assembles auditors with proven track records: finding critical vulnerabilities in public audit competitions.

Our audits are carried out by a curated team of the industry's top-performing security researchers, selected for your specific codebase, security needs, and budget.

Learn more about us at <https://zenith.security>.

1.2 Disclaimer

This report reflects an analysis conducted within a defined scope and time frame, based on provided materials and documentation. It does not encompass all possible vulnerabilities and should not be considered exhaustive.

The review and accompanying report are presented on an "as-is" and "as-available" basis, without any express or implied warranties.

Furthermore, this report neither endorses any specific project or team nor assures the complete security of the project.

1.3 Risk Classification

SEVERITY LEVEL	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

2

Executive Summary

2.1 About GMX Solana Protocol

GMX Solana is a decentralized spot and perpetual exchange that supports low swap fees and low price impact trades. Trading is supported by unique multi-asset pools that earns liquidity providers fees from market making, swap fees and leverage trading. Dynamic pricing is supported by Chainlink Oracles and an aggregate of prices from leading volume exchanges.

2.2 Scope

The engagement involved a review of the following targets:

Target	gmx-solana
---------------	------------

Repository	https://github.com/gmsol-labs/gmx-solana
-------------------	---

Commit Hash	0b92327a758f3efdc6a6fe7a692c1849f41aca60
--------------------	--

Files	Diff from cf310c8fc4e6b3e941913bc778e891e309830399
--------------	--

Target	Mitigation Review
---------------	-------------------

Repository	https://github.com/gmsol-labs/gmx-solana
-------------------	---

Commit Hash	665fe60a00e3134457b9bcea0b4d94c989a22f4f
--------------------	--

Files	Diff from cf310c8fc4e6b3e941913bc778e891e309830399
--------------	--

2.3 Audit Timeline

July 15, 2026	Audit start
July 17, 2026	Audit end
August 5, 2026	Report published

2.4 Issues Found

SEVERITY	COUNT
Critical Risk	0
High Risk	0
Medium Risk	1
Low Risk	5
Informational	5
Total Issues	11

3

Findings Summary

ID	Description	Status
M-1	Liquidation's allow_closed bypass makes newly accepted Unknown index reports actionable despite stale underlying prices	Acknowledged
L-1	The code upgrade includes no reconciliation for potentially orphaned pre-fix OI and borrowing state	Acknowledged
L-2	Re-enabling a market preserves a stale closed-market state	Acknowledged
L-3	v11 quotes can reach execution without mandatory independent plausibility or bid/ask freshness validation	Acknowledged
L-4	Feed-config mutations implicitly reset or rebind feed-specific safety policy	Acknowledged
L-5	v4 status overrides can make repeated closure prices continuously actionable while report observations advance	Resolved
I-1	Legacy price-feed accounts defer status policy until successfully refreshed after upgrade	Acknowledged
I-2	Coarse v11 status is always Unknown while adjacent API documentation describes conservative collapse	Resolved
I-3	Market-status policy is omitted from standard SDK serialization and CLI output	Resolved
I-4	Status flags can be stored for feeds that do not expose compatible market status	Acknowledged
I-5	extended_market_status documentation incorrectly states it is v11-only	Resolved

4

Findings

4.1 Medium Risk

A total of 1 medium risk findings were identified.

[M-1] Liquidation's `allow_closed` bypass makes newly accepted Unknown index reports actionable despite stale underlying prices

SEVERITY: Medium

IMPACT: High

STATUS: Acknowledged

LIKELIHOOD: Low

Target

- [crates/chainlink-datastreams/src/gmsol.rs#L138-L134](#)
- [crates/utls/src/price/feed_price.rs#L116-L160](#)
- [programs/store/src/states/oracle/feed.rs#L169-L183](#)
- [programs/store/src/instructions/exchange/position_cut.rs#L239-L248](#)
- [programs/store/src/instructions/exchange/position_cut.rs#L311-L323](#)
- [programs/store/src/ops/order.rs#L884-L896](#)
- [programs/store/src/states/market/utls.rs#L267-L280](#)

Description:

Before this change, `PriceFeedPrice::from_chainlink_report` rejected reports whose market status was `Unknown`:

```
MarketStatus::Unknown => {  
    return Err(crate::Error::UnknownMarketStatus);  
}
```

Such reports could not replace the stored price. The updated implementation instead stores the report with the legacy `Open` bit set and persists `MarketStatus::Unknown`.

For ordinary execution, `Unknown` resolves closed unless `AllowUnknown` is configured. The feed-reading API combines market status and underlying-mid freshness into a single `is_open` result:

```

if let MarketOpenness::Closed = self.market_status().openness(flags) {
    return false;
}

// ...

last_update_diff_secs
    ≤ market_close_timeout.saturating_sub(current_diff)

```

`last_update_diff_secs` is derived from `vll lastSeenTimestampNs` and represents the age of the underlying mid.

`PriceFeed::check_and_get_price` does not enforce this result when `allow_closed=true`:

```

let is_open = self.price.is_market_open(
    current,
    heartbeat_duration,
    feed_config.market_status_flags(),
);

if !allow_closed {
    require!(is_open, CoreError::MarketNotOpen);
}

let timestamp = self.price.ts();
if current > timestamp && current - timestamp > heartbeat_duration.into() {
    return Err(CoreError::PriceFeedNotUpdated.into());
}

```

The remaining heartbeat check uses `observationsTimestamp`. A report can therefore have a current observation envelope while its underlying mid has not changed recently.

Liquidation explicitly enables this bypass:

```

let (domain, allow_closed) = match kind {
    PositionCutKind::Liquidate => (DomainDisabledFlag::Liquidation, true),
    PositionCutKind::AutoDeleverage(_) =>
        (DomainDisabledFlag::AutoDeleveraging, false),
};

```

The bypass applies specifically to the market's index-token price. Liquidation continues requiring the long and short pool-token prices to be open:

```

MarketPriceOptions {

```

```
allow_index_closed: self.allow_closed,  
allow_long_closed: false,  
allow_short_closed: false,  
}
```

Consequently:

- a synthetic market with an RWA index and independently open collateral/pool-token feeds remains affected;
- a pure market that reuses the Unknown index token as its long or short pool token is generally blocked because the same token is read again with pool-token openness enforcement.

For example, assume a synthetic RWA market, open collateral-token feeds, a 30-second ordinary heartbeat, and a validly signed v11 index report with:

```
marketStatus          = Unknown  
observationsTimestamp = current time  
lastSeenTimestampNs   = current time - 10 minutes
```

The report can be published because Unknown is now accepted and the observation envelope is current. `is_market_open` correctly resolves false because the status is Unknown and the underlying mid is stale. Liquidation discards that verdict through `allow_closed=true`, while the observation-based heartbeat check passes.

The liquidation predicate is then evaluated using the stale index price. This can:

- force-close a position that is healthy at the current economic price;
- settle collateral and PnL at an unsuitable value;
- fail to liquidate a position that is economically insolvent;
- contribute to bad debt or unfair value transfer between traders and LPs.

The persistent `Market::Closed` safeguard does not prevent this. Its non-stale-oracle check compares the transaction oracle's observation timestamp against `closed_state_updated_at`. A newly published Unknown report has a current observation timestamp and therefore passes even if `lastSeenTimestampNs` is stale.

Signature verification, feed-ID binding, report expiry, timestamp monotonicity, and observation-age validation also pass. They establish authenticity and report-envelope recency, not underlying-mid freshness.

No `AllowUnknown` status flag is required because liquidation bypasses the complete `is_open` verdict.

The broad stale-price bypass already existed for reports categorized as known Closed. The scoped regression is that Unknown reports, which previously failed ingestion, can now reach the same path.

The complete preconditions are:

1. Chainlink publishes a valid Unknown report for the market's index token.
2. Its observationsTimestamp remains fresh while lastSeenTimestampNs is stale.
3. The market is synthetic, or its long and short pool-token feeds independently remain open.
4. PRICE_KEEPER publishes the report.
5. ORDER_KEEPER invokes liquidation.
6. The stale index price changes the liquidation result.

An unprivileged attacker cannot forge or publish the report and cannot invoke liquidation. This lowers likelihood but does not eliminate the potential economic impact.

Recommendations:

It is recommended to separate market-status permission from underlying-price age instead of representing both as one is_open boolean.

Unknown should not be treated like a known market closure:

- reject liquidation under Unknown; or
- require an independently validated fallback price.

For known Closed markets, liquidation may still need to remain available. Define a dedicated maximum age for a known closing price rather than requiring the ordinary freshness threshold unconditionally:

```
match status {
  MarketStatus::Unknown => {
    price = require_independent_fallback_price()?;
  }
  MarketStatus::Closed if allow_closed_status => {
    require!(
      underlying_age ≤ liquidation_closed_price_max_age,
      StaleUnderlyingPrice
    );
  }
  MarketStatus::Closed => {
    return err!(MarketNotOpen);
  }
  _ => {
    require!(
      underlying_age ≤ ordinary_price_max_age,
```

```
        StaleUnderlyingPrice
    );
    }
}
```

Additionally:

- preserve separate structured results for status, observation age, and underlying-mid age;
- make the permitted known-Closed liquidation age explicit and risk-based;
- ensure `Market :: Closed` staleness validation considers underlying update time;
- emit distinct errors for Unknown status and stale underlying price;
- test the synthetic-index configuration with open collateral-token feeds;
- test that a pure market remains blocked when the same Unknown token is also a pool token;
- test known Closed liquidation at, below, and above its dedicated staleness threshold;
- verify that the remediation preserves liquidation availability without making Unknown reports actionable.

References:

- [Chainlink RWA Advanced v11 Schema](#)
- [Chainlink Handling Market Events v11](#)
- [Chainlink Developer Responsibilities](#)

GMX: Acknowledged, for the following reasons: This issue is essentially about liquidations potentially being executed with stale prices. The same issue exists in the closed state (the suggested fix actually implicitly acknowledges this as well). Treating unknown as closed would reduce the time window in which the issue can occur, but that window is not the dominant part of the exposure compared with the closed case.

To actually resolve this issue, as the suggestion says, we would need to either introduce additional price sources or introduce an additional mandatory price expiration mechanism.

As for introducing additional price sources, this is an Oracle-layer problem, and we believe it should be solved at the Price Oracle Provider level (in fact, Chainlink is indeed actively working on solving it). At this stage, if the data quality does not meet the requirements, we can adopt more conservative parameters at the operational level to mitigate the issue, or even disallow holding positions during unknown / closed periods (which the current protocol is already capable of).

As for the mandatory price expiration mechanism, we already have a relatively complete design for it and will introduce it in a future version. The idea is that liquidations triggered by leverage can only be executed within a specified time window after the market is closed, while liquidations triggered by insufficient remaining collateral are not subject to this restriction (otherwise bad debt could be introduced). A partial mitigation that can be adopted at this stage is to enforce such restrictions off-chain at the keeper level (they cannot be enforced at the protocol level).

4.2 Low Risk

A total of 5 low risk findings were identified.

[L-1] The code upgrade includes no reconciliation for potentially orphaned pre-fix OI and borrowing state

SEVERITY: Low

IMPACT: Medium

STATUS: Acknowledged

LIKELIHOOD: Low

Target

- [crates/model/src/action/decrease_position/mod.rs#L291-L317](#)
- [crates/model/src/action/decrease_position/mod.rs#L604-L700](#)
- [crates/model/src/position.rs#L867-L885](#)

Description:

The upgrade fixes a rounding boundary where a partial long decrease could consume the position's entire `size_in_tokens` while consuming only part of its `size_in_usd`. The code-base, however, contains no migration for aggregate state potentially left by qualifying executions under earlier program versions.

Previously, the partial-close check promoted a decrease to a full close only when its remaining USD size was strictly below `min_position_size_usd`. Long decreases calculate the proportional token delta using ceiling division.

Let:

- s be the previous USD size;
- T be the previous token size;
- $D < s$ be the requested USD decrease; and
- $R = s - D$ be the remaining USD size.

The affected boundary is:

$$\text{ceil}(T \times D / S) = T$$

For a partial decrease, this implies:

$$R < S / T$$

The old minimum-size check prevents the issue whenever $R < \text{min_position_size_usd}$. Therefore, a successful affected execution additionally requires:

$$\text{min_position_size_usd} \leq R < S / T$$

In other words, the position's USD value per raw `size_in_tokens` unit must exceed the configured minimum position size.

This condition is unlikely for conventional assets with six to nine token decimals and a \$1 minimum. For example, one raw unit of an eight-decimal token priced at \$50,000 is worth only \$0.0005, so the minimum-size check would promote the close before the rounding boundary could corrupt accounting.

Nevertheless, the condition is legally reachable because the protocol supports synthetic token configurations with keeper-specified decimals and does not enforce:

$$\text{USD value of one raw index-token unit} \leq \text{min_position_size_usd}$$

For example, using $U = \text{MARKET_USD_UNIT}$, consider a sufficiently collateralized long position with no relevant price impact:

- $\text{size_in_usd} = 100 \times U$
- $\text{size_in_tokens} = 1$
- $\text{min_position_size_usd} = 50 \times U$
- $\text{size_delta_usd} = 50 \times U$

Then:

$$\text{ceil}(1 \times 50U / 100U) = 1$$

The remaining USD size equals the minimum, so the old strict comparison did not promote the request. The token dimension reached zero, causing the position to be removed:

```
should_remove =
  next_position_size_in_usd.is_zero()
  || next_position_size_in_tokens.is_zero();

if should_remove {
  *self.position.size_in_usd_mut() = Zero::zero();
  *self.position.size_in_tokens_mut() = Zero::zero();
  *self.position.collateral_amount_mut() = Zero::zero();
}
```

OI was nevertheless reduced only by the requested deltas:

```
self.position.update_open_interest(  
    &self.size_delta_usd.to_opposite_signed()?,  
    &execution.size_delta_in_tokens.to_opposite_signed()?,  
)?;
```

The market therefore retained $50 \times u$ of USD OI with no corresponding position or token OI.

The borrowing update similarly retained, when nonzero, the phantom contribution:

```
apply_factor(remaining_size_in_usd, next_borrowing_factor)
```

The target fix prevents new occurrences by promoting a decrease whenever its token delta would consume all remaining tokens. It cannot repair previous occurrences because the affected position has already been zeroed or closed. Subsequent incremental updates preserve any existing offset.

Potential consequences include distorted OI caps, funding, pending borrowing fees, pool PnL, market-token pricing, and deposit or withdrawal exchange rates.

The repository does not establish whether any deployed market satisfied the required granularity condition or experienced a qualifying execution. Consequently, this is a conditional upgrade-safety finding, not proof that deployed state is corrupted.

Recommendations:

Before introducing a repair, determine whether affected state was reachable in deployed configurations:

1. For every market and historical configuration interval running vulnerable code, compare `min_position_size_usd` against the maximum USD value represented by one raw index-token unit.
2. If that value was always less than or equal to the minimum position size, document the market as unaffected.
3. Otherwise, replay historical long decreases and identify removed positions satisfying:

```
actual_size_delta_usd < previous_size_in_usd  
ceil(previous_size_in_tokens × actual_size_delta_usd / previous_size_in_usd)  
== previous_size_in_tokens
```

For affected markets, pause value-changing operations and recompute `OI` and `TotalBorrowing` from active positions using the protocol's exact arithmetic. Apply any correction through an authenticated, version-gated, one-time migration.

Historical funding or market-token value transfers may not be reversible by correcting current aggregates alone. These effects should be quantified separately, with compensation considered where necessary.

If no qualifying execution is found, no state mutation should be performed.

GMX: Acknowledged. With `min_position_size_usd = $1`, the index-token price would need to exceed the following thresholds for one raw token unit to be worth more than the minimum position size: 5 decimals: $> \$100,000$; 6 decimals: $> \$1,000,000$; 8 decimals: $> \$100,000,000$; 9 decimals: $> \$1,000,000,000$. Current markets are far below these thresholds, so no state migration is indicated from current configuration alone.

[L-2] Re-enabling a market preserves a stale closed-market state

SEVERITY: Low

IMPACT: Medium

STATUS: Acknowledged

LIKELIHOOD: Low

Target

- [market.rs#215](#)

Description:

`unchecked_toggle_market` enables or disables a market by updating its `Enabled` flag.

However, enabling a market does not synchronize its cached `Closed` flag, while `update_closed_state` rejects disabled markets and therefore cannot keep that flag current during the disabled period.

A re-enabled market can use normal-market liquidation and borrowing parameters and skip the closed-price staleness restriction even though its index feed is closed; the inverse stale transition can leave an open market unusable.

Example Scenario:

1. A market is disabled at timestamp 1,000 while its cached `Closed` flag is `false`.
2. The index feed closes at timestamp 1,100, but `update_closed_state` cannot run because the market is disabled.
3. The keeper re-enables the market at timestamp 4,700, after a configured closed-price staleness limit of 300 seconds has elapsed.
4. A liquidation accepts the closed feed, but the cached `Closed = false` value bypasses closed-price staleness validation and selects normal-market risk parameters.

```
pub(crate) fn unchecked_toggle_market(ctx: Context<ToggleMarket>, enable:
    bool) -> Result<()> {
    ctx.accounts.market.load_mut()?.set_enabled(enable);
    Ok(())
}

require!(self.is_enabled(), CoreError::DisabledMarket);
```

```
if self.is_closed()
    && oracle.max_oracle_ts().saturating_add_unsigned(max_staleness)
        < self.closed_state_updated_at()?
{
    return Err(CoreError::OracleTimestampsAreSmallerThanRequired);
}
```

Recommendations:

When enabling a market, atomically refresh its closed state using authenticated current oracle data, or require a dedicated re-enable flow that cannot complete without this synchronization.

GMX: Acknowledged. This issue is related to the time-bounded consistency between the market closed state and the index price closed state discussed in L-2, and enforcing that consistency would mitigate it: if an operation is attempted after re-enabling while the market closed state and the index price closed state violate time-bounded consistency, the operation would fail until the market closed state is refreshed via a closed-state update.

For now, a simple and effective mitigation is to enable the market and update its closed state within the same transaction, which closes the window atomically. As an additional operational guideline, re-enabling during a window where the two states disagree should be avoided.

[L-3] v1l quotes can reach execution without mandatory independent plausibility or bid/ask freshness validation

SEVERITY: Low

IMPACT: Medium

STATUS: Acknowledged

LIKELIHOOD: Low

Target

- [crates/chainlink-datastreams/src/gmsol.rs#L9-L26](#)
- [crates/chainlink-datastreams/src/gmsol.rs#L45-L99](#)
- [crates/utils/src/token_config.rs#L20-L25](#)
- [programs/store/src/states/oracle/mod.rs#L432-L488](#)
- [programs/store/src/states/oracle/validator.rs#L61-L101](#)
- [programs/store/src/states/oracle/validator.rs#L141-L159](#)

Description:

v1l reports provide mid, bid, and ask. The protocol uses bid and ask as executable minimum and maximum prices but initially validates only that the values are non-negative and ordered:

```
if ask < price {  
    return Err(crate::Error::InvalidRange("ask < price"));  
}  
if price < bid {  
    return Err(crate::Error::InvalidRange("price < bid"));  
}
```

The codebase contains several relevant protections:

- Chainlink report signature verification;
- feed-ID and provider binding;
- report expiry and observation-age checks;
- monotonic observation timestamps;
- non-negative and ordered prices;
- downstream rejection of a zero minimum price;
- an optional bid/ask-to-mid deviation limit.

When a non-zero `max_deviation_factor` is configured, bid and ask are bounded relative to the same report's mid. If `AllowPriceAdjustment` is disabled, a value outside that bound is

rejected. If adjustment is enabled, it is clamped to $\text{mid} \pm \text{max_deviation}$ before execution:

```
if unit_prices.max.abs_diff(ref_price) > max_deviation {
    adjusted_price.get_or_insert(*price).max =
        price.max.with_unit_price(ref_price.checked_add(max_deviation)?,
            false)?;
}

if unit_prices.min.abs_diff(ref_price) > max_deviation {
    adjusted_price.get_or_insert(*price).min =
        price.min.with_unit_price(ref_price.checked_sub(max_deviation)?,
            true)?;
}
```

This protects against an excessively wide or asymmetric quote when properly configured. It does not fully address v1l suitability because:

1. `max_deviation_factor` defaults to disabled and is not required before enabling Extended or Overnight execution.
2. The reference is the same report's mid, not an independent price.
3. A uniformly shifted bid/mid/ask report remains within the configured relative bound.
4. `lastSeenTimestampNs` establishes freshness only for mid, not bid or ask.
5. Global independent reference-price validation is explicitly not implemented.

For example, if the independently correct price is 100, this signed report passes a 5% bid/ask-to-mid bound:

```
bid = 490
mid = 500
ask = 510
```

Bid and ask are each only 2% from the report's mid, although every value is approximately five times too high.

Separately, a report can carry a fresh mid alongside bid/ask values whose update time cannot be established from the v1l schema. Chainlink documents this timestamp limitation and warns that Extended/Overnight streams can contain carried-over quotes and single-provider anomalies.

An informed trader can use an authentic but economically unsuitable report to:

- close a pre-positioned long at an inflated value;
- open a short before normalization;

- perform the inverse operations on a low outlier;
- mint or burn market tokens against distorted NAV;
- trigger, avoid, or distort liquidation.

PnL caps, OI limits, reserves, fees, price impact, and acceptable-price bounds constrain exposure but do not validate absolute report correctness. An attacking user controls their own acceptable-price bounds, while LP NAV and liquidation victims are not protected by those bounds.

The attacker cannot forge this input. Exploitation requires Chainlink to publish an authentic anomalous report, successful publication by the price keeper, timely execution, and sufficient protocol exposure.

If GMX intentionally treats every verified Chainlink value as unconditionally authoritative, the uniformly wrong mid is an accepted oracle trust risk. However, Chainlink explicitly assigns application-level plausibility controls to integrators. The bid/ask component remains an integration concern because the protocol consumes fields whose freshness is not established by `lastSeenTimestampNs`.

Zero reports are not part of the value-extraction path. A zero minimum is rejected while constructing the transaction oracle, causing denial of service rather than execution at zero.

Recommendations:

It is recommended to require a non-zero, phase-specific `max_deviation_factor` before enabling Extended or Overnight v11 execution. The protocol should explicitly choose whether an excessive quote is rejected or conservatively clamped.

This existing control should be treated as a quote-to-mid spread bound, not independent outlier validation.

Additionally:

- implement the existing global reference-price deviation configuration;
- compare mid against a previous accepted value, bounded TWAP, or independent source;
- make these checks status-aware to accommodate legitimate session gaps, corporate actions, and stock splits;
- do not infer bid/ask freshness from `lastSeenTimestampNs`;
- where quote freshness cannot be independently established, pause affected execution, use a conservative validated-mid band, or obtain quotes from an auxiliary source;
- reject or mark zero reports unusable during ingestion and ensure the previous accepted price naturally becomes stale;
- configure separate Regular, Extended, and Overnight thresholds and transition procedures.

```
require!(
    feed_config.max_deviation_factor().is_some(),
    InvalidPriceConfiguration
);

validate_mid_against_status_aware_reference(
    report.mid,
    previous_accepted_mid,
    independent_reference,
    report.market_status,
)?;

let executable_price = validated_quote_or_conservative_mid_band(report)?;
```

References:

- [Chainlink RWA Advanced v11 Schema](#)
- [Chainlink 24/5 U.S. Equities User Guide](#)
- [Chainlink Developer Responsibilities](#)

GMX: Acknowledged. This issue cannot be fully addressed by the protocol alone; it requires coordination with the price provider. The current mitigation is to require an assessment of oracle data quality at the operations / risk-management level to decide whether the corresponding safeguards should be enabled. Once the price provider has specified the semantics in this area more explicitly, we will consider a more complete on-chain solution.

[L-4] Feed-config mutations implicitly reset or rebind feed-specific safety policy

SEVERITY: Low

IMPACT: Medium

STATUS: Acknowledged

LIKELIHOOD: Low

Target

- [programs/store/src/states/token_config.rs#L83-L113](#)
- [programs/store/src/instructions/token_config.rs#L330-L383](#)
- [crates/utls/src/token_config.rs#L300-L313](#)
- [crates/utls/src/token_config.rs#L382-L423](#)

Description:

Feed-specific safety policy should not change without explicit operator intent, particularly when the associated feed identity changes. The token-config mutation APIs violate this invariant in opposite ways.

When an existing token is updated through `push_to_token_map` or its synthetic equivalent, every `FeedConfig` is reconstructed:

```
self.feeds = feeds
    .into_iter()
    .zip(timestamp_adjustments.into_iter())
    .map(|(feed, timestamp_adjustment)| {

        FeedConfig::new(feed).with_timestamp_adjustment(timestamp_adjustment)
    })
    .collect::<Vec<_>>()
    .try_into()
    .map_err(|_| error!(CoreError::InvalidArgument))?;
```

`FeedConfig::new` resets both policy fields:

```
pub fn new(feed: Pubkey) → Self {
    Self {
        feed,
        timestamp_adjustment: DEFAULT_TIMESTAMP_ADJUSTMENT,
        max_deviation_ratio: DEFAULT_MAX_DEVIATION_RATIO,
```

```
        market_status_flags: Default::default(),
        reserved: Default::default(),
    }
}
```

DEFAULT_MAX_DEVIATION_RATIO is zero, meaning no deviation restriction. All-zero status flags permit RegularHours by default but reject the configurable off-hours statuses. Consequently, a bulk update can:

- remove HaltRegularHours, making the token-level oracle verdict permissive during regular hours;
- remove a configured maximum-deviation check;
- remove intended off-hours permissions and cause availability failures.

UpdateTokenConfigParams, including its conversion from an existing TokenConfig, cannot carry either field. Although the program logs that the fields were reset, it does not require acknowledgement or restoration.

Conversely, set_feed_config_v2 copies the existing FeedConfig before applying optional changes:

```
let previous_feed = *feed_config.feed();
let mut new_config = *feed_config;

if let Some(feed) = feed {
    new_config = new_config.with_feed(feed);
}
```

Changing only the feed ID therefore rebinds the old market-status flags, timestamp adjustment, and maximum-deviation factor to the replacement feed. For example, AllowClosed or an off-hours permission configured for one stream can remain active after replacing it with a stream having different schema or session semantics.

Both transitions require the MARKET_KEEPER role. They do not directly let an unprivileged attacker bypass feed identity, signature, freshness, or market-closure validation. Material loss additionally requires an unsafe operator transition, relevant oracle conditions, and subsequent value-changing execution. Nevertheless, an attacker may capitalize on the resulting policy state.

The maximum-deviation reset predates the scoped change. The addition of market-status flags extends this existing behavior to session policy and introduces policy inheritance when a feed ID is replaced.

Recommendations:

It is recommended to make feed-policy transitions explicit and atomic in a versioned instruction:

- Compare each provider's old and new feed identity independently.
- Preserve policy by default only when the provider and feed ID remain unchanged.
- Require a complete replacement policy whenever the feed ID or expected provider changes.
- If compatibility cannot be verified on-chain, do not permit policy preservation across a feed-ID change.
- Include market-status flags and maximum-deviation values in the versioned update parameters.
- Emit a structured old/new-policy event and add regression tests for unchanged feeds, feed replacement, and provider changes.

```
enum FeedPolicyUpdate {
  Preserve, // Valid only when provider and feed ID are unchanged.
  Replace {
    market_status_flags: u8,
    max_deviation_factor: Option<u128>,
  },
}
```

GMX: Acknowledged. As already noted in the code, this is a known issue. Improving the logic here would require a systematic rework of the feed config update rules, which is too large in scope at this stage.

[L-5] v4 status overrides can make repeated closure prices continuously actionable while report observations advance

SEVERITY: Low

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Low

Target

- [crates/chainlink-datastreams/src/report.rs#L224-L243](#)
- [crates/chainlink-datastreams/src/gmsol.rs#L38-L99](#)
- [crates/utils/src/price/market_status.rs#L34-L43](#)
- [crates/utils/src/price/feed_price.rs#L116-L161](#)
- [programs/store/src/states/oracle/feed.rs#L169-L184](#)
- [programs/store/src/states/market/utils.rs#L288-L304](#)

Description:

The market-status policy documentation states that freshness is a hard floor: a stale price should remain closed regardless of any configured status flag.

This guarantee relies on `last_update_diff`, which is available only when the report schema provides an underlying-price update timestamp.

v4 reports provide a market status but no last-update timestamp:

```
Ok(Report {  
  feed_id: report.feed_id,  
  observations_timestamp: report.observations_timestamp,  
  last_update_timestamp: None,  
  price,  
  bid: price,  
  ask: price,  
  market_status,  
  extended_market_status: Some(market_status.into()),  
  // ...  
})
```

Consequently, conversion does not enable `LastUpdateDiffEnabled` for v4. When `AllowClosed` or `AllowUnknown` changes the reported status from closed to open, `is_market_open` reaches the following branch:

```
let Some(last_update_diff_secs) = self.last_update_diff_secs() else {
    return true;
};
```

There is no remaining economic-price age check inside `is_market_open`.

`PriceFeed::check_and_get_price` separately checks `observationsTimestamp`:

```
let timestamp = self.price.ts();
if current > timestamp && current - timestamp > heartbeat_duration.into() {
    return Err(CoreError::PriceFeedNotUpdated.into());
}
```

This proves that the report envelope is recent. It does not establish when the v4 economic price last changed.

Chainlink documents that RWA streams can repeat the closing price in subsequent reports while a market remains closed. When new closure reports continue advancing `observationsTimestamp`, the observation heartbeat passes even though the economic price remains frozen.

A complete reachable sequence is:

1. A token uses a v4 RWA feed.
2. `MARKET_KEEPER` enables `AllowClosed` or `AllowUnknown`.
3. The underlying market closes or enters an indeterminate state.
4. Chainlink continues producing reports with an advancing `observationsTimestamp` and a repeated price.
5. `PRICE_KEEPER` continues publishing those reports.
6. The market remains open from the previous session, or `ORDER_KEEPER` refreshes its persistent `Closed` state using the same overridden oracle-openness result.
7. An informed trader executes against the frozen price after off-market information changes its fair value.

The persistent `Market::Closed` flag does not independently correct this. Its update path derives the flag from `oracle.is_open(index_token)`:

```
let is_closed = !oracle.is_open(&self.meta.index_token_mint)?;
self.set_closed(is_closed)?;
```

If a Closed/Unknown v4 price resolves open only because of the configured override, an ORDER_KEEPER refresh can also mark the market open using that same result.

An informed trader can then:

- open or close a position against a stale closing price;
- mint or burn market tokens against distorted NAV;
- transfer value from LPs around news, weekends, halts, or delayed openings;
- contribute to bad debt if the market gaps materially before correction.

Existing protections limit but do not eliminate the path:

- default all-zero status policy keeps Closed and Unknown closed;
- only MARKET_KEEPER can enable the override;
- signatures, feed IDs, report expiry, timestamp monotonicity, and observation age are validated;
- fees, price impact, OI caps, reserves, and slippage limits constrain exposure.

None of these establishes when a v4 price last changed.

The exposure is not automatically unbounded. If closure reports stop advancing `observationsTimestamp`, the ordinary heartbeat eventually rejects the stored report. Potentially continuous exposure requires Chainlink to continue producing and PRICE_KEEPER to continue publishing fresh report envelopes containing the repeated price.

This report-publication behavior is an external condition rather than a guarantee established by the program. Chainlink's general RWA documentation describes repeated closing prices in subsequent reports, but no deployed v4 feed or keeper configuration has been demonstrated.

If GMX intentionally accepts trading at the last close whenever `AllowClosed` is set, the remaining defect is that it cannot enforce how old that closing price is. The same override can authorize a recently closed price or a price that has remained frozen for days, despite the documentation claiming that freshness always remains a hard floor.

For v8 and v11, an underlying update timestamp exists. Their related exposure is bounded by:

```
current time - last economic update <= heartbeat_duration
```

A large configured heartbeat can still create a material window, but it is a bounded sub-observation rather than the core v4 issue.

Recommendations:

It is recommended to reject `AllowClosed` and `AllowUnknown` for v4 feeds when the flag is configured.

The schema version is encoded in the configured Chainlink feed ID and should be validated during:

- status-flag changes;
- feed-address changes;
- provider changes; and
- token-map activation.

```
if matches!(flag, AllowClosed | AllowUnknown) && enable {
    require!(
        configured_feed.schema_version().supports_underlying_update_timestamp(),
        MissingUnderlyingUpdateTimestamp
    );
}
```

Add a read-time defense in depth so incompatible persisted or inherited configurations also fail closed:

```
if status_override_opens_market(status, flags) {
    require!(
        price.last_update_diff_secs().is_some(),
        MissingUnderlyingUpdateTimestamp
    );
}
```

Additionally:

- migrate v4 RWA feeds requiring Closed/Unknown operation to v8/v11;
- treat Unknown separately and require an independently validated fallback;
- for v8/v11, use a dedicated underlying-price age limit independent of ordinary report heartbeat;
- prevent Market:: from being cleared when a Closed/Unknown status resolves open through an override but the schema lacks economic-freshness evidence;
- do not prevent normal v4 Open reports from clearing Market::;
- correct the documentation to describe the v4 limitation and bounded v8/v11 behavior;
- test flag setting, feed replacement, inherited policy, and market-state updates across v4, v8, and v11;
- test continued closure-report publication with advancing observation time and a frozen economic price.

References:

- [Chainlink Handling Market Events](#)
- [Chainlink Data Streams Market Hours](#)

- [Chainlink Developer Responsibilities](#)

GMX: Resolved with [PR-396](#) by removing v4 support.

Zenith: Verified.

4.3 Informational

A total of 5 informational findings were identified.

[I-1] Legacy price-feed accounts defer status policy until successfully refreshed after upgrade

SEVERITY: Informational

IMPACT: Informational

STATUS: Acknowledged

LIKELIHOOD: Low

Target

- [crates/utils/src/price/feed_price.rs#L19-L29](#)
- [crates/utils/src/price/feed_price.rs#L116-L170](#)
- [crates/chainlink-datastreams/src/gmsol.rs#L102-L109](#)
- [programs/store/src/instructions/oracle/custom.rs#L13-L35](#)
- [programs/store/src/states/oracle/feed.rs#L148-L184](#)
- [crates/sdk/src/client/pull_oracle.rs#L66-L99](#)
- [crates/sdk/src/client/chainlink/pull_oracle/pull_oracle_impl.rs#L275-L317](#)

Description:

Before the upgrade, the byte now used for `market_status_value` was explicitly zeroed padding. Every legacy `PriceFeedPrice` therefore deserializes with status value zero, representing `MarketStatus::Disabled`.

`Disabled` skips status-policy resolution and defers to the legacy `PriceFlag::Open` bit. This preserves the pre-upgrade verdict:

- a legacy `Open` report remains open while otherwise fresh;
- a legacy `Closed` report retains `open=false` and remains closed.

However, flags configured after the upgrade do not affect a legacy stored price until a successful post-upgrade report overwrites it with a canonical status.

The relevant safety case is `HaltRegularHours`. If enabled before refreshing a fresh legacy `Open` price, that account continues to resolve open temporarily. Permissive off-hours flags fail in the opposite direction because they cannot override a legacy `open=false` report.

The window is bounded by the old report's remaining observation-heartbeat lifetime and

any legacy last-update freshness restriction. Once stale, `check_and_get_price` rejects it.

A logical feed can have multiple persistent `PriceFeed` accounts because the account address includes authority and index information. Refreshing one account does not migrate every account that could later be supplied by an authorized custom controller.

The standard SDK substantially mitigates this condition. Its pull-oracle flow fetches reports, updates the specific price-feed accounts selected for the operation, and appends the consuming operation afterward:

```
let (instructions, map) = self
    .pull_oracle
    .fetch_price_update_instructions(&self.price_updates, options.clone())
    .await?;

// ...

tx.append(consume, false)?;
```

After a successful standard SDK update, the selected account contains post-upgrade status data before consumption.

The on-chain program does not require this ordering or verify that the supplied account was written by the current program version. An authorized custom keeper can consume another still-fresh legacy account whose status remains `Disabled`.

No unprivileged attacker controls this sequence, and `allow_closed` paths already ignore status policy. This is an Informational migration-ordering concern rather than an independently exploitable vulnerability.

The concern applies to status-bearing Chainlink schemas such as v4, v8, and v11. Status-less schemas remain `Disabled` after refresh and require separate policy compatibility handling.

Recommendations:

It is recommended to disable affected markets while completing one of the following migration strategies:

1. Inventory every consumable `PriceFeed` account associated with status-bearing feeds.
2. Successfully refresh each account and verify that its stored status is no longer `Disabled`; or
3. Wait until every legacy account exceeds its configured heartbeat and can no longer be consumed.
4. Enable the intended status policy only after completing and verifying the migration.

5. Re-enable the affected markets.

The standard SDK should remain the preferred execution path because it refreshes the selected account before consumption. Custom keeper implementations should enforce the same ordering and verify post-update account state.

For an on-chain guarantee, add an account-level status-initialization or layout-version marker set by the post-upgrade writer. When non-default status policy is configured, an account lacking that marker should fail closed. This prevents authorized consumers from accidentally reintroducing an unmigrated account.

Additionally:

- Document the migration procedure and multiple-account consideration.
- Add upgrade tests with multiple serialized legacy accounts.
- Add deployment preflight and post-migration readback.
- Reject status flags for schemas that can never produce market status.

GMX: Acknowledged. Since market status flags should only be enabled for v11 feeds in the foreseeable future, and no v11 feeds are currently enabled, we will only record this issue in our internal documentation.

[I-2] Coarse v11 status is always Unknown while adjacent API documentation describes conservative collapse

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- [crates/chainlink-datastreams/src/report.rs#L40-L79](#)
- [crates/chainlink-datastreams/src/report.rs#L102-L114](#)
- [crates/chainlink-datastreams/src/report.rs#L283-L305](#)

Description:

At the comparison merge base, v11 reports were conservatively converted to the coarse `MarketStatus` representation:

- `RegularHours` → `Open`
- `Unknown` → `Unknown`
- `PreMarket`, `PostMarket`, `Overnight`, and `Closed` → `Closed`

The target decoder instead stores `Unknown` for every v11 status:

```
// v11 has no coarse status of its own; force the deprecated
// field to Unknown so consumers use `extended_market_status`.
market_status: MarketStatus::Unknown,
extended_market_status: Some(extended),
```

The deprecated `market_status()` method's current documentation only calls the result "coarse." However, the adjacent `extended_market_status()` documentation still describes `market_status()` as a compatibility API that collapses non-regular-hours states to `Closed`:

```
/// Downstream consumers can use this for finer-grained trading decisions
/// instead of relying on [Self::market_status()], which is a
/// compatibility
/// trade-off that collapses all non-regular-hours states to
/// [MarketStatus::Closed].
```

This description contradicts the current implementation. Tests also contain comments stating that `PreMarket` and status 5 should map to `Closed` while asserting that the coarse result

is Unknown.

A consumer first adopting v11 support while retaining the following permissive check would accept every v11 phase:

```
report.market_status() ≠ MarketStatus::Closed
```

A strict `MarketStatus::Open` comparison remains safe. Genuine Unknown reports also passed the permissive check under the previous mapping.

This is not a released same-input compatibility regression:

- published v0.9.0 does not support v11;
- target v0.10.0-beta.1 is not currently published on crates.io;
- no in-repository production caller uses the deprecated coarse API.

Only direct source/git consumers of an intermediate development revision could observe changed behavior for the same v11 input. For other consumers, this is a prospective integration and documentation hazard when first adopting v11.

The adjacent documentation has another mismatch: it says `extended_market_status()` is “v11 only,” while the target decoder also returns canonical `Some( ... )` values for v4 and v8.

Recommendations:

It is recommended to restore the conservative coarse mapping before publishing v11 support:

```
let market_status = match extended {
    ExtendedMarketStatus::Unknown ⇒ MarketStatus::Unknown,
    ExtendedMarketStatus::RegularHours ⇒ MarketStatus::Open,
    ExtendedMarketStatus::PreMarket
    | ExtendedMarketStatus::PostMarket
    | ExtendedMarketStatus::Overnight
    | ExtendedMarketStatus::Closed ⇒ MarketStatus::Closed,
};
```

This preserves a safe compatibility representation while directing integrations that require phase-specific behavior to `extended_market_status()`.

Additionally:

- Update `extended_market_status()` documentation to state that it returns canonical status for v4, v8, and v11.
- Add table-driven tests covering coarse and extended output for every v11 status.
- Correct contradictory test comments.

- Document that consumers interpreting coarse status as openness must require `MarketStatus::Open`.
- Prefer the extended API for every new integration.
- If conservative coarse semantics are not desired, redesign or remove the coarse API before release rather than returning `Unknown` for every `v11` state behind contradictory documentation.

GMX: Resolved with [PR-397](#) by documenting that `v11` returns `Unknown` and directing consumers to the extended-status API.

Zenith: Verified.

[I-3] Market-status policy is omitted from standard SDK serialization and CLI output

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- [crates/utils/src/token_config.rs#L369-L376](#)
- [crates/sdk/src/serde/serde_token_map.rs#L68-L100](#)
- [crates/cli/src/commands/market.rs#L425-L499](#)
- [programs/store/src/lib.rs#L903-L935](#)

Description:

FeedConfig exposes its market-status bitmap through `market_status_flags()`, but the standard serializable representation omits it:

```
pub struct SerdeFeedConfig {
    pub feed_id: String,
    pub feed_id_encoding: Encoding,
    pub timestamp_adjustment: u32,
    pub max_deviation_factor: Option<Value>,
}
```

The CLI token-config views convert configurations into `SerdeTokenConfig`. Consequently, neither machine-readable output nor detailed/list views expose the raw bitmap or decoded status flags.

The policy is still available on-chain. Custom clients can load the token-map account and call `FeedConfig::market_status_flags()`. The direct setter also logs the token, provider, changed flag, previous value, and new value. Therefore, the policy is not fundamentally hidden; it is omitted specifically from the standard serialization and inspection workflow.

No dedicated structured event is emitted for policy changes. Such an event could improve historical indexing, but its absence does not affect policy correctness.

This issue does not create an unsafe state or attacker-controlled transition. It is an Informational operator-observability concern that can make configuration review and incident response less convenient.

Recommendations:

It is recommended to add the raw bitmap to `SerdeFeedConfig` while retaining backward-compatible deserialization:

```
pub struct SerdeFeedConfig {
    pub feed_id: String,
    pub feed_id_encoding: Encoding,
    pub timestamp_adjustment: u32,
    pub max_deviation_factor: Option<Value>,
    #[cfg_attr(serde, serde(default))]
    pub market_status_flags: u8,
}
```

Populate the field with:

```
config.market_status_flags().into_value()
```

Additionally:

- Display decoded flag names in detailed CLI output.
- Preserve and display the raw bitmap so unknown future bits are not lost.
- Include the bitmap in machine-readable configuration output and comparisons.
- Add serialization tests for zero, individual, combined, and unknown flag bits.
- Add an optional post-write readback that verifies expected policy.
- Optionally emit a dedicated event containing token map, token, provider, feed ID, and old/new bitmap for indexers.

Structured events should supplement account-state readback rather than replace it.

GMX: Resolved with [PR-390](#).

Zenith: Verified.

[I-4] Status flags can be stored for feeds that do not expose compatible market status

SEVERITY: Informational	IMPACT: Informational
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- [programs/store/src/lib.rs#L884-L935](#)
- [crates/chainlink-datastreams/src/gmsol.rs#L115-L134](#)
- [crates/chainlink-datastreams/src/report.rs#L183-L305](#)
- [programs/store/src/states/oracle/mod.rs#L405-L424](#)

Description:

The market-status setter permits flags to be stored for any configured provider without verifying that the provider or report schema can produce the corresponding status.

For non-Chainlink providers, the program emits a warning but still persists the flag and returns success:

```
if !matches!(kind, PriceProviderKind::ChainlinkDataStreams) {
    msg!("set market status flag: note: this provider does not report
    market status; the flag has no effect for now");
}

let previous = SetFeedConfigMarketStatusFlag::invoke_unchecked(
    ctx,
    &kind,
    market_status_flag,
    enable,
)?;
```

The effective capabilities differ by provider and schema:

- Pyth and Switchboard do not expose market status, so all flags are ineffective.
- Chainlink v2, v3, and v7 canonicalize to Disabled, so all flags are ineffective.
- Chainlink v4 and v8 expose only Unknown, Regular/Open, and Closed; phase-specific PreMarket, PostMarket, and Overnight flags are ineffective.
- Chainlink v11 exposes granular phases, but the applicable mapping depends on the

feed's trading-hours model.

The on-chain program, SDK, and CLI do not reject incompatible combinations. Non-Chainlink writes are warned about, while incompatible Chainlink schema/flag combinations receive no equivalent warning.

This does not provide an unprivileged attack path: only `MARKET_KEEPER` can write the configuration, and the instruction correctly stores the requested flag. It is principally an operator-safety and configuration-visibility concern. Latent policy may also be intentional preconfiguration rather than a defect.

Chainlink guarantees that v11 values 1, 3, and 4 do not occur on compliant standard-hours feeds. Their interpretation is therefore not a current exploit path. The universal decoder must continue accepting them because they are valid for 24/5 feeds.

Recommendations:

It is recommended to make ineffective or pending policy explicit:

- Reject enabling flags for non-Chainlink providers unless the caller explicitly requests pending/pre-staged policy.
- Decode the configured Chainlink feed's schema version and reject flags that the schema can never emit.
- Warn or require confirmation when the currently stored price has `Disabled` status.
- Add equivalent validation to the SDK and CLI.
- Display whether each configured flag is currently effective, ineffective, or pending.
- Preserve context-neutral v11 decoding; apply standard-hours versus 24/5 restrictions only after the feed model is known.
- Emit structured policy-change events for monitoring.

A runtime rule that closes every status-less feed carrying a non-default policy should be introduced only if the protocol explicitly guarantees that every stored policy must be immediately enforceable, because it would otherwise change the behavior of intentionally pre-staged configurations.

GMX: Partially Resolved with [PR-395](#) and acknowledged. Unsupported providers are blocked, but incompatible Chainlink schemas can still accept ineffective flags.

Zenith: Verified, and acknowledged.

[I-5] `extended_market_status` documentation incorrectly states it is v11-only

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- [report.rs#108](#)

Description:

`Report::extended_market_status` exposes the granular market status decoded from report schemas that carry market-status data.

However, its documentation states that the value is available only for v11 reports even though v4 and v8 now also populate `extended_market_status`.

The stale documentation can cause integrators and reviewers to incorrectly assume that v4 and v8 return `None` instead of a converted granular status.

```
/// Returns extended market status (v11 only).
pub fn extended_market_status(&self) → Option<ExtendedMarketStatus> {
    self.extended_market_status
}
```

Recommendations:

Update the documentation to state that the method returns a granular status for every report schema that defines market status, including v4, v8, and v11.

```
/// Returns extended market status (v11 only).
/// Returns extended market status (v4, v8, v11 only).
```

GMX: Resolved with [PR-397](#).

Zenith: Verified.